

Unsupervised Learning Masterclass: Numerical Walkthroughs

A hand-calculable pedagogical guide tracing the core pillars of unsupervised learning.

1. K-Means Clustering: Step-by-Step Walkthrough

K-Means is an iterative, partition-based clustering algorithm. It groups data points into **K** distinct clusters by minimizing the distance between points and their respective cluster centers (centroids). It runs in a loop of **Assignment** (assigning points to the nearest centroid) and **Update** (moving centroids to the average center of their assigned points) until convergence.

The Sample Dataset

Imagine we have 4 retail customers with two features: Age and Spending Score (1-100):

- **Point A:** Age = 20, Spending = 80 (Young High Spender)
- **Point B:** Age = 22, Spending = 75 (Young High Spender)
- **Point C:** Age = 60, Spending = 20 (Senior Budget Shopper)
- **Point D:** Age = 62, Spending = 22 (Senior Budget Shopper)

We want to group them into **K = 2 clusters**.

Iteration 1: Starting the Loop

Step 1: Initial Centroid Placement

We randomly choose two points to act as our starting cluster centers:

- Centroid 1 (\$C_1\$): Placed at Point A (**20, 80**)

- Centroid 2 (\$C_2\$): Placed at Point C (**60, 20**)

Step 2: Distance & Cluster Assignment

We calculate the straight-line (Euclidean) distance from every point to both centroids:
$$\text{Distance} = \sqrt{(X_{\text{point}} - X_{\text{centroid}})^2 + (Y_{\text{point}} - Y_{\text{centroid}})^2}$$

- **Point A (20, 80):** Dist to \$C_1\$ is 0. Dist to \$C_2\$ is 72.1. **Verdict: Cluster 1.**
- **Point B (22, 75):** Dist to \$C_1\$ is $\sqrt{(22-20)^2 + (75-80)^2} = \sqrt{29}$ pprox 5.39\$. Dist to \$C_2\$ is 66.85. **Verdict: Cluster 1.**
- **Point C (60, 20):** Dist to \$C_1\$ is 72.1. Dist to \$C_2\$ is 0. **Verdict: Cluster 2.**
- **Point D (62, 22):** Dist to \$C_1\$ is 71.61. Dist to \$C_2\$ is $\sqrt{(62-60)^2 + (22-20)^2} = \sqrt{8}$ pprox 2.83\$. **Verdict: Cluster 2.**

Step 3: Recalculate Centroid Positions (Centroid Update)

Centroids shift to the exact average center of their assigned members:

- **New Centroid 1 (\$C_1'\$)** (Average of A and B): Age = $\frac{20+22}{2} = 21$ \$, Spending = $\frac{80+75}{2} = 77.5$ \$. Coordinates: (**21, 77.5**).
- **New Centroid 2 (\$C_2'\$)** (Average of C and D): Age = $\frac{60+62}{2} = 61$ \$, Spending = $\frac{20+22}{2} = 21$ \$. Coordinates: (**61, 21**).

Iteration 2: Verifying Convergence

Step 4: Recalculate Distances & Assignments

We check if any points change their cluster labels when measured against the shifted centroids \$C_1'\$ (21, 77.5) and \$C_2'\$ (61, 21):

- **Point A (20, 80):** Dist to \$C_1'\$ is $\sqrt{(20-21)^2 + (80-77.5)^2} = 2.69$ \$. Dist to \$C_2'\$ is 71.85. Remains in **Cluster 1**.
- **Point B (22, 75):** Dist to \$C_1'\$ is 2.69. Dist to \$C_2'\$ is 67.90. Remains in **Cluster 1**.
- **Point C (60, 20):** Dist to \$C_1'\$ is 69.90. Dist to \$C_2'\$ is $\sqrt{(60-61)^2 + (20-21)^2} = 1.41$ \$. Remains in **Cluster 2**.
- **Point D (62, 22):** Dist to \$C_1'\$ is 69.11. Dist to \$C_2'\$ is 1.41. Remains in **Cluster 2**.

Since no points changed their cluster assignments, the algorithm has officially **converged**!

Customer	Age, Spend	Final Cluster	Demographic Segment
Point A	(20, 80)	Cluster 1	Young High Spenders
Point B	(22, 75)	Cluster 1	Young High Spenders
Point C	(60, 20)	Cluster 2	Senior Budget Shoppers
Point D	(62, 22)	Cluster 2	Senior Budget Shoppers

2. DBSCAN Clustering: Step-by-Step Walkthrough

DBSCAN groups points based on local density. It does not require you to pre-specify the number of clusters, and it is capable of discovering clusters of arbitrary shapes (like concentric rings) while cleanly filtering out outliers. It relies on a search radius ϵ (**Epsilon**) and a neighborhood threshold **MinPts**.

The Sample Dataset

Imagine we have 5 points plotted on a 2D coordinate grid:

- **Point A:** (1, 1)
- **Point B:** (1, 2)
- **Point C:** (2, 1)
- **Point D:** (1, 10)
- **Point E:** (10, 10)

We set $\epsilon = 3$ and **MinPts** = 2.

Step-by-Step Execution

Round 1: Scanning Point A (1, 1)

1. Select Point A as our start point.
2. Scan neighbors within distance $\epsilon = 3$: B (1, 2) and C (2, 1) are inside. D and E are outside.
3. Count neighborhood size: Neighbors of A are {A, B, C} (3 points).
4. Evaluate density: Since the count (3) is $\geq$ MinPts (2), **Point A is labeled a Core Point**.
5. Form cluster: A starts **Cluster 1**. Its unvisited neighbors {B, C} are added to the expansion queue.

Round 2: Expanding via Point B (1, 2)

1. Pop Point B from the queue.
2. Scan neighbors: Neighbors are {A, B, C} (3 points).
3. Evaluate density: Since count $\geq$ MinPts, **Point B is also a Core Point**.
4. Expand: B joins Cluster 1.

Round 3: Expanding via Point C (2, 1)

1. Pop Point C from the queue.

- 2. Scan neighbors: Neighbors are {A, B, C} (3 points).
- 3. Evaluate density: Since count $\geq$ MinPts, **Point C is a Core Point**.
- 4. Expand: C joins Cluster 1. The queue is now empty. Cluster 1 is complete!

Round 4: Scanning Point D (1, 10)

- 1. Select Point D (next unvisited point).
- 2. Scan neighbors: The only neighbor of D is {D} (1 point). Distance to E (10, 10) is 9.0 (Outside).
- 3. Evaluate density: Since the count (1) is $<$ MinPts (2), Point D is not a Core Point.
- 4. Label status: Since D has no connections to Core points, it is labeled as **Noise (Outlier)**.

Round 5: Scanning Point E (10, 10)

- 1. Select Point E (last unvisited point).
- 2. Scan neighbors: Only {E} (1 point).
- 3. Evaluate density: Since count $<$ MinPts, Point E is not a Core Point.
- 4. Label status: Point E is labeled as **Noise (Outlier)**.

Point	Coordinates	Neighbors Found	Class Type	Final Assignment
Point A	(1, 1)	3	Core Point	Cluster 1
Point B	(1, 2)	3	Core Point	Cluster 1
Point C	(2, 1)	3	Core Point	Cluster 1
Point D	(1, 10)	1	Outlier	Noise (Isolated Outlier)
Point E	(10, 10)	1	Outlier	Noise (Isolated Outlier)

3. PCA Dimensionality Reduction: Step-by-Step Walkthrough

Principal Component Analysis (PCA) simplifies high-dimensional data by projecting it onto a lower-dimensional subspace (e.g., compressing 2D down to 1D) while keeping the maximum amount of original spread (variance) of the shape.

The Sample Dataset

Imagine we have 3 points plotted on a 2D coordinate grid:

- **Point A:** (1, 2)
- **Point B:** (3, 4)
- **Point C:** (5, 6)

These points form a diagonal line. We want to compress them into a single **1D coordinate**.

Step-by-Step Mathematical Compression

Step 1: Mean Centering

We shift the data so its average center rests exactly at (0, 0).

1. Calculate averages: Mean X = $\frac{1+3+5}{3} = 3$, Mean Y = $\frac{2+4+6}{3} = 4$.
2. Subtract the means:
 - **Point A'** = $(1 - 3, 2 - 4) = \mathbf{(-2, -2)}$
 - **Point B'** = $(3 - 3, 4 - 4) = \mathbf{(0, 0)}$
 - **Point C'** = $(5 - 3, 6 - 4) = \mathbf{(2, 2)}$

Step 2: Compute the Covariance Matrix

Using the sample covariance formula (dividing by $N-1 = 2$):

1. $\text{Cov}(X, X) = \frac{(-2)^2 + 0^2 + 2^2}{2} = \mathbf{4}$
2. $\text{Cov}(Y, Y) = \frac{(-2)^2 + 0^2 + 2^2}{2} = \mathbf{4}$

3. $\text{Cov}(X, Y) = \frac{(-2 \times -2) + (0 \times 0) + (2 \times 2)}{2} = 4$

Our Covariance Matrix (\$\$) is:
$$S = \begin{pmatrix} 4 & 4 \\ 4 & 4 \end{pmatrix}$$

Step 3: Find Eigenvalues & Eigenvectors

1. Solve $\det(S - \lambda I) = 0$: $\det \begin{pmatrix} 4 - \lambda & 4 \\ 4 & 4 - \lambda \end{pmatrix} = 0 \implies \lambda^2 - 8\lambda = 0$ This yields: **$\lambda_1 = 8$** (PC1, captures 100% of variance) and **$\lambda_2 = 0$** (PC2).
2. Solve for the eigenvector $\mathbf{v}$ for $\lambda_1 = 8$: $\begin{pmatrix} -4 & 4 \\ 4 & -4 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = 0 \implies x = y$ Normalizing this vector gives: $\mathbf{v} = \begin{pmatrix} 0.7071 \\ 0.7071 \end{pmatrix}$

Step 4: Projection to 1D

We calculate the dot product of centered coordinates with $\mathbf{v}$: $\text{Projected Coordinate} = (X_{\text{centered}} \times 0.7071) + (Y_{\text{centered}} \times 0.7071)$

- Point A' (-2, -2):** $(-2 \times 0.7071) + (-2 \times 0.7071) = -2.83$
- Point B' (0, 0):** $(0 \times 0.7071) + (0 \times 0.7071) = 0.00$
- Point C' (2, 2):** $(2 \times 0.7071) + (2 \times 0.7071) = 2.83$

Original 2D Point	Centered 2D Point	Compressed 1D Coordinate	Information Retained
Point A (1, 2)	(-2, -2)	-2.83	100% (No loss)
Point B (3, 4)	(0, 0)	0.00	100% (No loss)
Point C (5, 6)	(2, 2)	2.83	100% (No loss)

4. Isolation Forest Outliers: Step-by-Step Walkthrough

Isolation Forest isolates anomalies directly by recursively slicing the data space with random lines. Because anomalies are sparse and far away from the crowd, they get isolated in very few slices (shallow tree depth), whereas normal points require many slices to separate.

The Sample Dataset

Imagine we are auditing Credit Card Transactions with two features: Spend Amount (\$) and Distance from Home (km). We have 4 transactions (3 normal, 1 outlier):

- **Point A:** Spend = \$20, Distance = 2 km (Normal)
- **Point B:** Spend = \$25, Distance = 5 km (Normal)
- **Point C:** Spend = \$30, Distance = 3 km (Normal)
- **Point X:** Spend = \$500, Distance = 80 km (Anomaly Outlier)

Step-by-Step Space Slicing

Round 1: The First Split

1. Pick random feature: The algorithm randomly chooses **Spend Amount**.
2. Find range: The minimum spend is \$20 (Point A) and maximum is \$500 (Point X).
3. Pick random split value: It picks a random number between \$20 and \$500. Let's say it picks **\$150**.
4. Divide the dataset:
 - Left Branch (Spend < \$150): Contains Point A, Point B, Point C.
 - Right Branch (Spend ≥ \$150): Contains Point X.

🌟 **Point X is now completely alone. It is isolated in just 1 split! (Path Length = 1)**

Round 2: Splitting the Left Branch

We look only at the remaining group {A, B, C} on the left branch:

1. Pick random feature: It randomly chooses **Distance from Home**.
2. Find range: Min = 2 km, Max = 5 km.

- 3. Pick random split value: It picks a random number between 2 and 5. Let's say it picks **4 km**.
- 4. Divide the remaining dataset:
 - Left Sub-Branch (Distance < 4 km): Contains Point A (2 km) and Point C (3 km).
 - Right Sub-Branch (Distance ≥ 4 km): Contains Point B (5 km).

🌟 **Point B is now completely alone. It is isolated after 2 splits! (Path Length = 2)**

Round 3: Splitting the Last Group

Only Point A and Point C are left together:

- 1. Pick random feature: It chooses **Spend Amount**.
- 2. Find range: Min = \$20, Max = \$30.
- 3. Pick random split value: It randomly picks **\$26**.
- 4. Divide the dataset:
 - Left Sub-Branch (Spend < \$26): Contains Point A (\$20).
 - Right Sub-Branch (Spend ≥ \$26): Contains Point C (\$30).

🌟 **Point A and Point C are now both isolated. (Path Length = 3 for both)**

Data Point	Transaction Details	Splits to Isolate (Path Length)	Verdict
Point X	Anomaly (\$500, 80km)	1 split (Very Shallow)	 HIGH RISK FRAUD FLAG
Point B	Normal (\$25, 5km)	2 splits	 Approved (Normal)
Point A	Normal (\$20, 2km)	3 splits (Deep)	 Approved (Normal)
Point C	Normal (\$30, 3km)	3 splits (Deep)	 Approved (Normal)

5. Apriori Association Rules: Step-by-Step Walkthrough

The Apriori algorithm is used for Market Basket Analysis to discover purchase relationships between products. It operates on the principle that: *If an itemset is frequent, then all of its subsets must also be frequent.* This allows the algorithm to prune (discard) unpopular item combinations early, saving massive amounts of computation. It evaluates rules using **Support**, **Confidence**, and **Lift**.

The Sample Dataset

Imagine our supermarket database has exactly 3 customer receipts (transactions):

- **Receipt 1 (T1):** {Bread, Butter}
- **Receipt 2 (T2):** {Bread, Butter, Milk}
- **Receipt 3 (T3):** {Milk}

We configure Apriori with a **Minimum Support of 50%** (must appear in ≥ 2 receipts) and a **Minimum Confidence of 60%**.

Step-by-Step Association Mining

Step 1: Candidate Single Items (\$C_1\$)

We list individual items and count their support:

- **{Bread}**: 2 receipts (T1, T2) $\rightarrow$ Support = $\$ \frac{2}{3} = \mathbf{66.7\%}$ (Passes!)
- **{Butter}**: 2 receipts (T1, T2) $\rightarrow$ Support = $\$ \frac{2}{3} = \mathbf{66.7\%}$ (Passes!)
- **{Milk}**: 2 receipts (T2, T3) $\rightarrow$ Support = $\$ \frac{2}{3} = \mathbf{66.7\%}$ (Passes!)

All items become **Frequent 1-Itemsets (\$L_1\$)**.

Step 2: Candidate Item Pairs (\$C_2\$)

We combine items from \$L_1\$ into pairs and count support:

- **{Bread, Butter}**: 2 receipts (T1, T2) $\rightarrow$ Support = $\$ \frac{2}{3} = \mathbf{66.7\%}$ (Passes!)
- **{Bread, Milk}**: 1 receipt (T2) $\rightarrow$ Support = $\$ \frac{1}{3} = \mathbf{33.3\%}$ ( **Pruned! Below 50%**)

Only **{Bread, Butter}** becomes a **Frequent 2-Itemset (\$L_2\$)**.

Step 3: Rule Generation & Metric Audits

We extract rules from our frequent pair {Bread, Butter}:

- **Rule A: $[Bread] \rightarrow [Butter]$**
 - Support (Popularity): Support of {Bread, Butter} = $\mathbf{66.7\%}$
 - Confidence (Reliability): $\frac{\text{Support}(Bread, Butter)}{\text{Support}(Bread)} = \frac{2}{2} = \mathbf{100\%}$ (Passes ≥ 60%!)
 - Lift (Strength): $\frac{\text{Confidence}}{\text{Support}(Butter)} = \frac{1.00}{0.667} = \mathbf{1.5x}$ (Positive correlation!)
- **Rule B: $[Butter] \rightarrow [Bread]$**
 - Support: $\mathbf{66.7\%}$
 - Confidence: $\frac{\text{Support}(Bread, Butter)}{\text{Support}(Butter)} = \frac{2}{2} = \mathbf{100\%}$ (Passes!)
 - Lift: $\frac{1.00}{0.667} = \mathbf{1.5x}$

Discovered Rule	Rule Popularity (Support)	Rule Reliability (Confidence)	Strength Multiplier (Lift)	Retail Recommendation
[Bread] → [Butter]	66.7%	100%	1.5x	Place Bread and Butter on the same aisle to drive convenience buys.
[Butter] → [Bread]	66.7%	100%	1.5x	Bundle them together in a "Breakfast Special" package promotion.